

Code Refactoring Exercise : The Professor Class
Polish the `professor.h` and `professor.cpp` to meet a higher standard. Upload those two files or pastes the content of the two files.

Refactor Professor to meet these requirements:

Full Encapsulation

All data members private

Public getters: `getId()`, `getName()`, `getCourseCount()`,

`getCourse(index)` (all const)

`setName()` with validation (reject empty strings)

Proper Constructors

`Professor()` — default ID=0, name="Unknown"

`Professor(int id, const string& name)` — must use initialization list

Validation:

if `id <= 0` or `name.empty()`, use defaults and print warning

Initialize all array elements to " "

Destructor

Print "Professor [name] destroyed\n" to observe object lifetime

Static Constants

Replace 50 with static const `int MAX_COURSES = 50`

Const Correctness

`print()` and all getters must be const

Reference parameters where appropriate: `const string&`

```
#ifndef PROFESSOR_H
#define PROFESSOR_H

#include <string>
using namespace std;

class Professor {
public:
    int id;
    string name;
    string courses[50];
    int courseCount;

    Professor();

    void print() const;
    void addCourse(const string& courseCode);
    void removeCourse(const string& courseCode);
};

#endif // PROFESSOR_H
```

```
#pragma once
```

```
#include
```

```
using namespace std;
```

```
class Professor {
```

```
private:
```

```
    int id;
```

```
    string name;
```

```
    string courses[50];
```

```
    int courseCount;
```

```
    static const int MAX_COURSES = 50;
```

```
professor.h :
```

```
#ifndef PROFESSOR_H
```

```
#define PROFESSOR_H
```

```
#include
```

```
class Professor {
```

```
private:
```

```
    static const int MAX_COURSES = 50;
```

```
    int id;
```

```
    std::string name;
```

```
    std::string courses[MAX_COURSES];
```

```
    int courseCount;
```

Static member variables

```
// ===== Declaration (Header .h) =====
class ChessPlayer {
private:
    static int totalPlayers; // Declaration only, not definition
    string name;
public:
    ChessPlayer(string n) : name(n) {
        totalPlayers++; // Increment count for each new player
    }
    static int getTotal() { return totalPlayers; }
};

// ===== Definition (Implementation .cpp) =====
// ⚠ MUST define outside the class! Most easily forgotten part!
int ChessPlayer::totalPlayers = 0; // Definition and initialization
```

```
// ❌ Error 1: Forgetting to define outside the class
// If you only declare but don't define, linker error: undefined reference

// ❌ Error 2: Repeating 'static' in the out-of-class definition
int ChessPlayer::static totalPlayers = 0; // Wrong! No static keyword needed

// ❌ Error 3: Initializing static variable in constructor
ChessPlayer(string n) : name(n) {
    totalPlayers = 0; // Wrong! Resets to zero every construction—defeats
    counting
}

// ✅ Correct: Initialize only once outside the class
int ChessPlayer::totalPlayers = 0; // Initialized once at program startup
```

const member variables

```
class ChessPiece {
private:
    const string pieceID;      // Each piece has a unique ID
    const bool isWhite;        // Color cannot change once determined
    int positionX, positionY;

public:
    // ✅ MUST initialize const members in the initializer list!
    ChessPiece(string id, bool white, int x, int y)
        : pieceID(id)          // const members MUST be initialized here
        , isWhite(white)        // Cannot assign inside function body!
        , positionX(x)
        , positionY(y)
    {
        // ❌ Cannot assign inside constructor body
        // pieceID = id; // Compile error: const member cannot be modified
    }
};
```

Static const member variables

```
class ChessGame {
private:
    //  C++11 onwards: integral types can initialize directly in class
    static const int BOARD_SIZE = 8;           // Integral: in-class init
    static const int MAX_PLAYERS = 2;         // Integral: in-class init

    //  Non-integral: must initialize outside class before C++17
    static const string GAME_NAME;             // Declaration

public:
    void printBoard() {
        // Use static constant
        for(int i = 0; i < BOARD_SIZE; i++) {
            // ...
        }
    }
};

// Non-integral static const MUST be defined outside class (pre-C++17)
const string ChessGame::GAME_NAME = "International Chess";
```

```
class Professor : public Person {  
private:  
    string courses[50];  
    int courseCount;  
    const int MAX_COURSES;
```

```
class Professor : public Person {  
private:  
    string courses[50];  
    int courseCount;  
    static int MAX_COURSES = 0;
```

```
class Professor : public Person {  
private:  
    string courses[50];  
    int courseCount;  
    static const int MAX_COURSES = 0;
```

```
class Professor : public Person {  
private:  
    string courses[50];  
    int courseCount;  
    static const double MAX_COURSES = 50;  
    // anything wrong? why it can be initialized here?
```

Key points

- **Static Members**

Don't forget to define outside the class, all objects share one copy Never initialize in constructor, declaration and definition must separate

- **Const Members**

Initializer list is their home, cannot assign in function body One per object independent value, fixed forever after creation

- **Static Const**

Integral types C++11 in-class fine, non-integral pre-C++17 outside define Shared + read-only is the nature, class-level constants most reliable

C++ OBJECT-ORIENTED
PROGRAMMING

All about a family! Inheritance and UML

Building Family Trees in Code

How classes inherit traits like children inherit genes — a comprehensive guide to creating reusable, maintainable code through object-oriented design



UML: The Architect's Blueprint



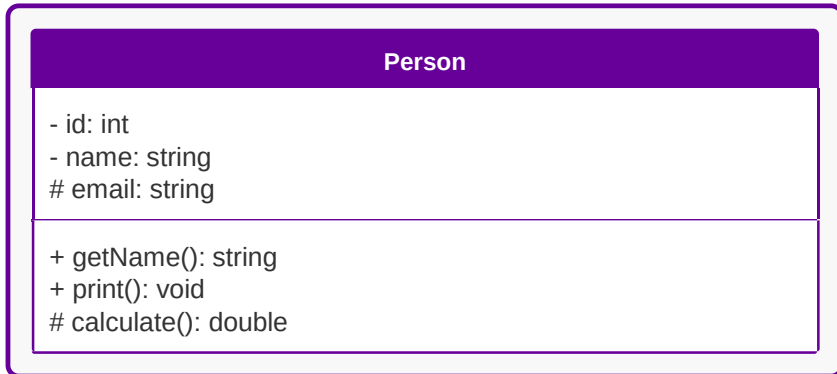
The Blueprint Analogy

Analogy: Before building a house, architects create blueprints showing rooms, walls, and wiring. They don't pour concrete immediately! Similarly, **UML (Unified Modeling Language)** is a visual blueprint for software. It lets us design classes and relationships before writing code.

Key Benefits

- 1 Visual Communication** — Show structures without code details
- 2 Design Before Coding** — Plan architecture first
- 3 Documentation** — Understand existing code quickly

UML Class Notation



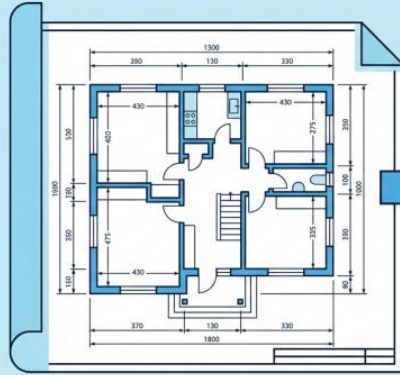
Access Modifiers

- + public** — Everyone can access
- private** — Only this class
- # protected** — This class + derived classes



UML is to code what blueprints are to buildings

UML Diagram



Source Code



ClassName

- id: int
- name: string
- # email: string
- + getName(): string
- # print(): void
- calculate(): double

- + public
- private
- # protected

```
class Person {  
protected:  
    int id;  
    string name;  
  
public:  
    Person(int id, const string& name);  
    ~Person() = default; // Destructor (non-v  
  
    int getId() const { return id; }  
    string getName() const { return name; }  
    void print() const;  
};
```

Classes Deep Dive: Beyond the Basics

Key Objectives

- Master advanced class logic
- Transition to reliable systems
- Architect extensible codebases

Prerequisites:

Basic C++ Syntax & Compilation

C++ OBJECT-ORIENTED CONCEPTS

Constructor



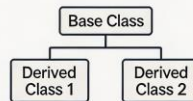
Initializes object creation

Destructor



Cleans up object memory

Inheritance



Derives parent class features

Polymorphism



One interface, multiple forms

The Four Pillars of Advanced OOP

Inheritance: Modeling 'is-a' Relationships

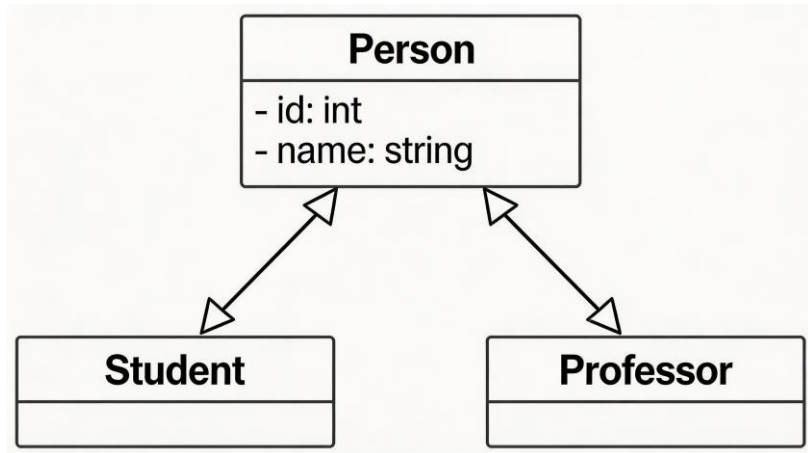
The 'is-a' Concept

- Derived class *is a* base class
- Example: A Student **is a** Person

```
class Student : public Person
```

Code Reuse:

Common attributes (id, name) move to base class, reducing redundancy.



Access Control:

- **Public:** Accessible by everyone
- **Protected:** Accessible by derived classes
- **Private:** Base class only

The Constructor Chain in Inheritance

1. Base Class



2. Derived Class

Syntax: Initialization List

```
Professor(id, name) : Person(id, name) {  
    // Base runs before these braces  
}
```

Key Implementation Rules

- Use initialization list for base
- Default base constructor called implicitly if omitted
- Ensure all base fields are initialized

Types of Relationships

01

Inheritance: "Is-A" / 继承: "是一个"

A student is a person (inherits person attributes)

class Student : public Person

02

Composition: "Has-A" (Strong) / 组合: "有一个" (强)

A human has a heart

The heart cannot exist independently. If the human dies, the heart dies too

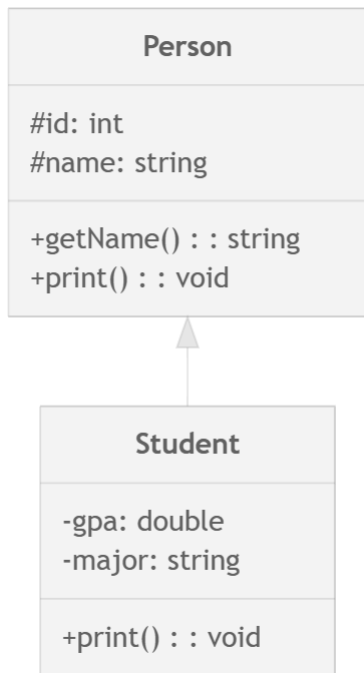
03

Aggregation: "Has-A" (Weak) / 聚合: "有一个" (弱)

A university has students.

But students exist independently If the university closes, students still exist

Inheritance: "Is-A" / 继承: "是一个"

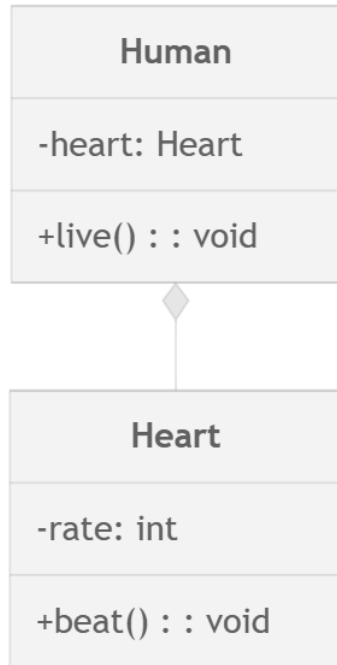


UML Notation: Hollow triangle arrow pointing to parent

•C++: `class Student : public Person`

Any other examples?

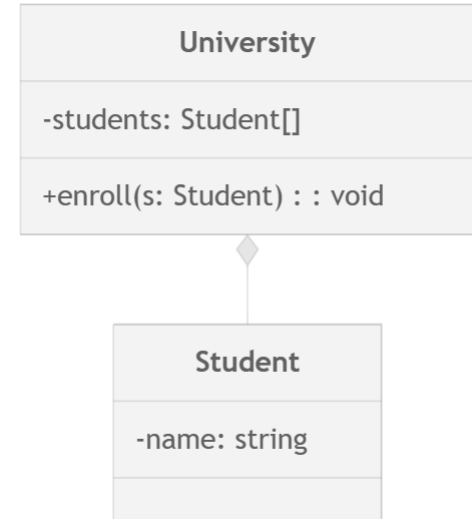
Composition vs Aggregation



UML Notation: Filled diamond

C++:

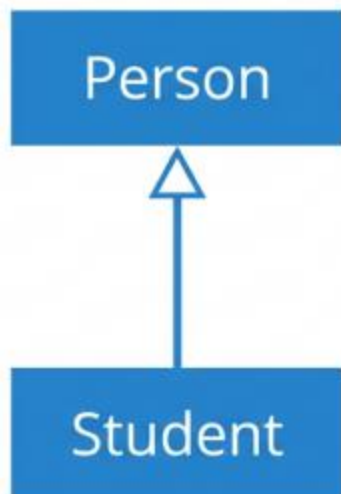
```
class Human {
    Heart heart;
}; (member object)
```



UML Notation: Hollow diamond

```
class University {
    Student* students[];
}; (pointers)
```


Inheritance



is-a

Composition



has-a (strong) - dies together

Aggregation



has-a (weak) - independent

Design a university system with Department, Professor, and Person.

Department
-name: string -faculty: Professor[]
+addProfessor() :: void +printRoster() :: void

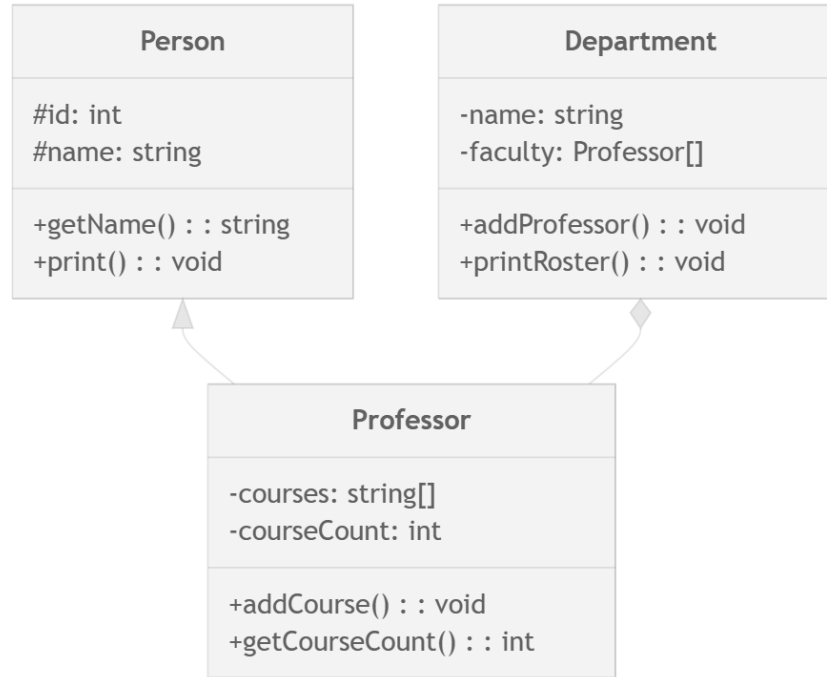
Professor
-courses: string[]
+addCourse() :: void

Person
#id: int #name: string
+getName() :: string +print() :: void

作答

Professor is-a Person (inheritance).

Department has Professors (aggregation - professors exist outside the department).



Inheritance in C++

```
class Student {
private:
    int id;           // Duplicated!
    string name;      // Duplicated!
    string email;     // Duplicated!
    double gpa;       // Student-specific
public:
    void print() {    // Similar logic
        cout << "Student: " << name << endl;
    }
};

class Professor {
private:
    int id;           // Duplicated again!
    string name;      // Duplicated again!
    string email;     // Duplicated again!
    string dept;      // Professor-specific
public:
    void print() {    // Similar logic
        cout << "Professor: " << name << endl;
    }
};
```

```
class Person {
protected: // ← Key: accessible to children (derived classes)
    int id;
    string name;
    string email;

public:
    Person(int id, const string& name, const string& email)
        : id(id), name(name), email(email) {}

    void printBasic() const {
        cout << "[" << id << "]" " << name << endl;
    }
};

// Derived class = inherits + specializes
class Student : public Person { // "Student is-a Person"
private:
    double gpa;           // Student-specific only
    string major;

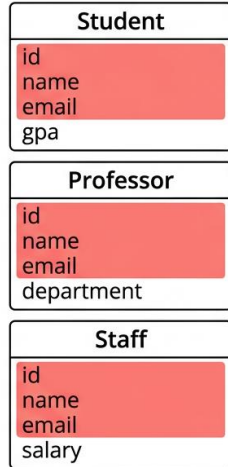
public:
    Student(int id, const string& name, const string& email,
            double gpa, const string& major)
        : Person(id, name, email) // ← Call parent's constructor!
        , gpa(gpa), major(major) {}

    void print() const {
        printBasic(); // ← Reuse parent's method!
        cout << " Major: " << major << ", GPA: " << gpa << endl;
    }
};
```

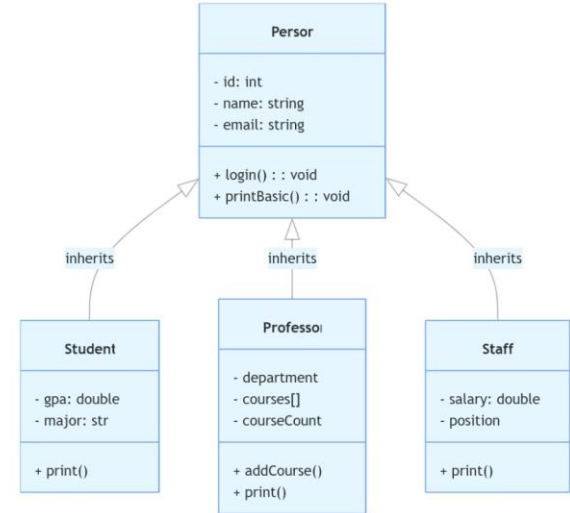
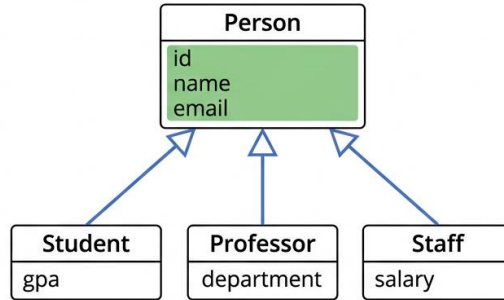
Why inheritance?

COMPARISON: CODE DUPLICATION VS. INHERITANCE SOLUTION

WITHOUT INHERITANCE



WITH INHERITANCE



```

class Person {
protected: // + Key: accessible to children (derived classes)
    int id;
    string name;
    string email;

public:
    Person(int id, const string& name, const string& email)
        : id(id), name(name), email(email) {}

    void printBasic() const {
        cout << "[" << id << "]" " << name << endl;
    }
};

// Derived class = inherits + specializes
class Student : public Person { // "Student is-a Person"
private:
    double gpa; // Student-specific only
    string major;

public:
    Student(int id, const string& name, const string& email,
            double gpa, const string& major)
        : Person(id, name, email) // + Call parent's constructor!
        , gpa(gpa), major(major) {}

    void print() const {
        printBasic(); // + Reuse parent's method!
        cout << " Major: " << major << ", GPA: " << gpa << endl;
    }
};

```

PUBLIC

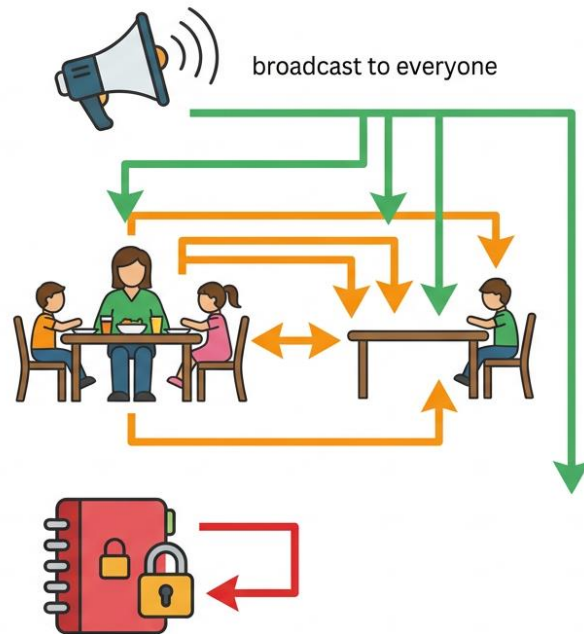
Visible and accessible by all parts of the system.

PROTECTED

Shared information within related classes or family members.

PRIVATE

Strictly encapsulated and hidden from outside access.



Public vs. Private Inheritance

Base Access	Public Inherit.	Private Inherit.
public	public	private
protected	protected	private

Public Inheritance

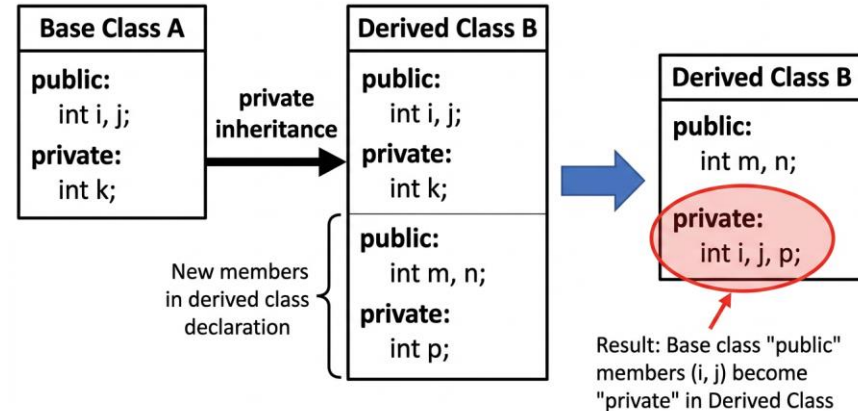
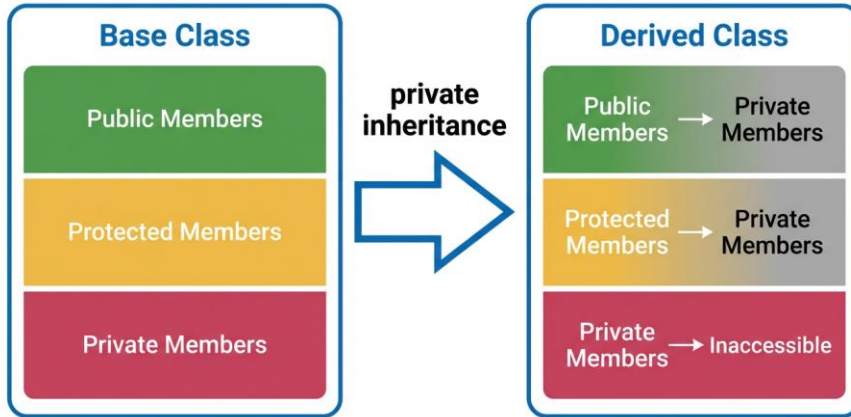
- Models "is-a" relationship
- Supports polymorphism
- Preserves base interface

Private Inheritance

- "Implemented-in-terms-of"
- Implementation reuse only
- Hides base interface

Guideline: Prefer Public for domain modeling; Private for hidden implementation details.

Private Inheritance



Protected Inheritance

```
class A {  
protected:  
    int x;  
};  
  
int main() {  
    A a;  
    a.x = 5;    //Is it correct?  
}
```

```
class A {  
protected:  
    int x;  
};  
class B: public A{  
public:  
    void function();  
};  
void B::function() {  
    x = 5;    //Correct  
}
```

To design a geometry system, what should be the relation between class point and rectangle?

- ☐ A Inheritance
- ☐ B Composition
- ☐ C Aggregation
- ☐ D Hard to tell

提交

Points and rectangles

Rectangle.h - Derived class using private inheritance:

Point.h - Base class defining a 2D point:

```
#ifndef _POINT_H
#define _POINT_H

class Point { // Base class Point definition
public: // Public function members
    void initPoint(float x = 0, float y = 0) {
        this->x = x;
        this->y = y;
    }
    void move(float offX, float offY) {
        x += offX;
        y += offY;
    }
    float getX() const { return x; }
    float getY() const { return y; }
private: // Private data members
    float x, y;
};

#endif // _POINT_H
```

```
#ifndef _RECTANGLE_H
#define _RECTANGLE_H
#include "Point.h"

class Rectangle : private Point { // Derived class definition
public: // New public function members
    void initRectangle(float x, float y, float w, float h) {
        initPoint(x, y); // Call base class public member function
        this->w = w;
        this->h = h;
    }
    void move(float offX, float offY) {
        Point::move(offX, offY); // Explicitly call base class move
    }
    float getX() const { return Point::getX(); } // Expose via wrapper
    float getY() const { return Point::getY(); } // Expose via wrapper
    float getH() const { return h; }
    float getW() const { return w; }
private: // New private data members
    float w, h; // Width and height
};

#endif // _RECTANGLE_H
```

Main.cpp - Using the Rectangle class:

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    Rectangle rect; // Create a Rectangle object
    rect.initRectangle(2, 3, 20, 10); // Set rectangle position and size
    rect.move(3, 2); // Move rectangle position

    cout << "The data of rect(x, y, w, h): " << endl;
    cout << rect.getX() << ", " // Access via wrapper methods
        << rect.getY() << ", "
        << rect.getW() << ", "
        << rect.getH() << endl;
    return 0;
}
```

Output:

The data of rect(x, y, w, h):
5, 5, 20, 10

Friend

Imagine you have:

- **Private bedroom** (private members): Only you can enter
- **Protected living room** (protected members): Family can enter
- **Public yard** (public members): Anyone can enter

Now imagine your **best friend** or **trusted neighbor**:

- They're not family (not inheritance)
- But you give them a **spare key** to your home
- They can enter your **private bedroom** when needed



Friend Function: External Function with Private Access

```
class BankAccount {
private:
    double balance;    // Private - only the account can modify
    string password;   // Private - sensitive data

public:
    BankAccount(double initial) : balance(initial) {}

    // Regular public interface
    double getBalance() const { return balance; }

    // Declare external function as FRIEND
    // 声明外部函数为友元
    friend bool transferFunds(BankAccount& from, BankAccount& to, double amount);
    friend class BankManager; // Entire class is a friend!
};

// External function - but can access private members!
// 外部函数 - 但可以访问私有成员!
bool transferFunds(BankAccount& from, BankAccount& to, double amount) {
    // This function is NOT a member of BankAccount
    // 这个函数不是BankAccount的成员
    // But it can access PRIVATE members!
    // 但它可以访问私有成员!

    if (from.balance >= amount) { // ✓ Accessing private 'balance'!
        from.balance -= amount;
        to.balance += amount;    // ✓ Accessing private 'balance'!
        return true;
    }
    return false;
}
```

Friend Class: Entire Class Gets Access

```
class BankManager; // Forward declaration

class BankAccount {
private:
    double balance;
    int accountNumber;

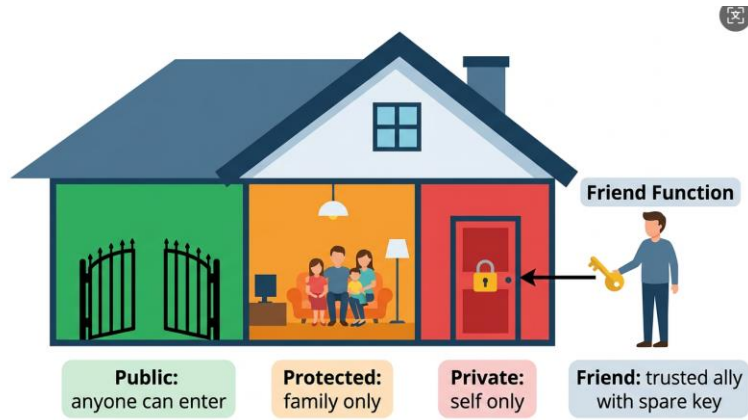
public:
    BankAccount(int num, double initial)
        : accountNumber(num), balance(initial) {}

    // Grant ALL members of BankManager access to ALL private members
    // 授予BankManager所有成员访问所有私有成员的权限
    friend class BankManager;
};

class BankManager {
public:
    // Can access private data of BankAccount directly!
    // 可以直接访问BankAccount的私有数据!
    void auditAccount(const BankAccount& account) {
        cout << "Account #" << account.accountNumber << endl; // ✓ Private!
        cout << "Balance: $" << account.balance << endl;      // ✓ Private!
    }

    void freezeAccount(BankAccount& account) {
        // Direct modification of private data!
        account.balance = 0; // ✓ Can modify private!
        cout << "Account frozen!" << endl;
    }
};
```

Key Characteristics of Friend



Feature	Description	Analogy
Not a member	Friend functions are NOT class members	Friend has a spare key, not a bedroom
Not symmetric	If A is B's friend, B is NOT automatically A's friend	You gave them a key, they didn't give you one
Not inherited	Derived classes don't inherit friendship	Children don't inherit parent's friends
Declared in class	Friend declaration goes inside the class	You decide who gets a key
Accesses all private	Friends can access ALL private/protected members	Spare key opens all doors

Constructor Order: Building a Family

Phase	Order	Memory Analogy
Construction	Base → Members → Derived	Build foundation first
Destruction	Derived → Members → Base	Remove roof before foundation

When building a custom home:

1.Foundation (Base class) must be poured first

2.Framing (Members) comes next

3.Finishing touches (Derived class) comes last

When demolishing:

1.Finishing touches removed first

2.Framing removed next

3.Foundation removed last (reverse order!)

```
class Person {
public:
    Person() { cout << "1. Person constructor (Foundation)" << endl; }
    ~Person() { cout << "4. Person destructor (Last)" << endl; }
};

class Student : public Person {
    int studentId;
public:
    Student() : Person() { // Explicit base call
        cout << "2. Student constructor (Finishing)" << endl;
    }
    ~Student() {
        cout << "3. Student destructor (First)" << endl;
    }
};

// Usage:
Student s;

// Output:
// 1. Person constructor (Foundation)
// 2. Student constructor (Finishing)
// 3. Student destructor (First)
// 4. Person destructor (Last)
```

CONSTRUCTOR & DESTRUCTOR ORDER IN INHERITANCE

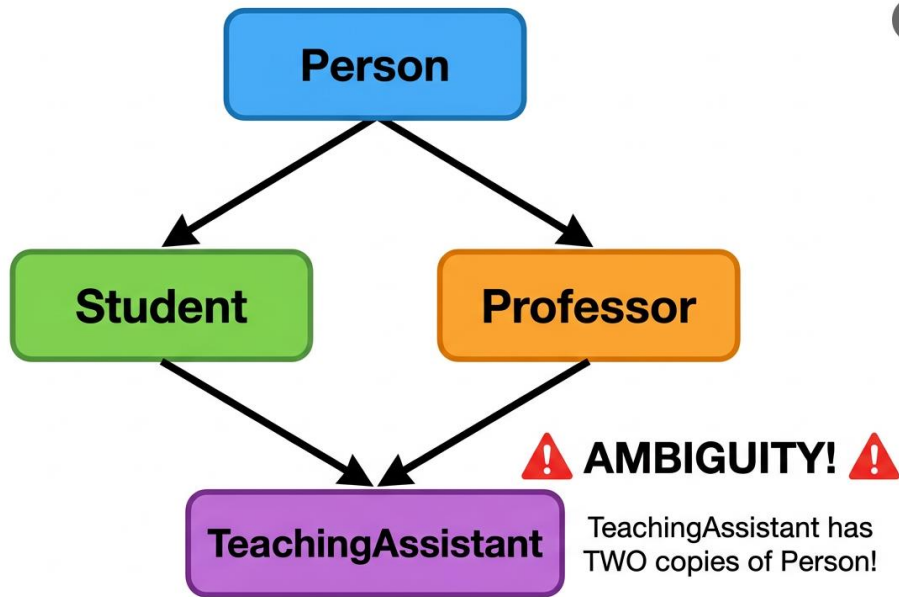
CONSTRUCTION (Build) 



DESTRUCTION (Demolish) 



The Diamond Problem / 菱形问题



```
class Person {
public:
    string name;
    Person(const string& n) : name(n) {}
};

class Student : public Person { // ← Inherits Person
public:
    double gpa;
    Student(const string& n, double g) : Person(n), gpa(g) {}
};

class Professor : public Person { // ← Also inherits Person!
public:
    string department;
    Professor(const string& n, const string& d) : Person(n), department(d) {}
};

// Teaching Assistant is both Student AND Professor
class TA : public Student, public Professor {
public:
    TA(const string& n, double g, const string& d)
        : Student(n, g), Professor(n, d) // ⚠️ PROBLEM: Person constructed TWICE!
    {}
};

// What happens here?
TA ta("Alice", 3.8, "CS");
// ta.name = ??? // ERROR: ambiguous! Which name? Student's or Professor's?
```

Virtual Base Class

```
class Person {
public:
    string name;
    int id;
    Person(const string& n, int i) : name(n), id(i) {}
};

// Use VIRTUAL inheritance
class Student : virtual public Person { // ← virtual!
public:
    double gpa;
    Student(const string& n, int i, double g)
        : Person(n, i), gpa(g) {}
};

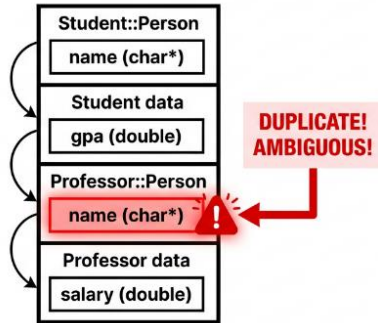
class Professor : virtual public Person { // ← virtual!
public:
    string department;
    Professor(const string& n, int i, const string& d)
        : Person(n, i), department(d) {}
};
```

```
// Now TA has only ONE Person part
class TA : public Student, public Professor {
public:
    // MUST initialize virtual base directly!
    TA(const string& n, int i, double g, const string& d)
        : Person(n, i)           // ← Initialize virtual base FIRST
        , Student(n, i, g)       // Virtual base init ignored here
        , Professor(n, i, d)     // Virtual base init ignored here
    {}
};

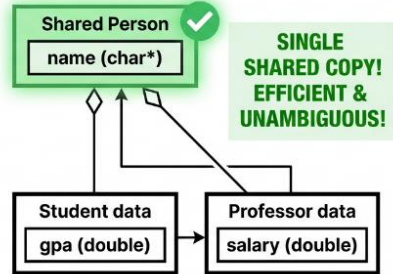
// Usage:
TA ta("Alice", 1001, 3.8, "CS");
cout << ta.name;           // ✓ OK: Unambiguous! Single copy
cout << ta.id;             // ✓ OK: Only one id
```

DIAGRAM: MEMORY LAYOUT COMPARISON – MULTIPLE INHERITANCE

WITHOUT virtual (BROKEN)



WITH virtual (FIXED)



TA Object in Memory:

Shared Person part ├ name: "Alice" └ id: 1001	← name, id (SINGLE copy!)
Student-specific part └ gpa: 3.8	
Professor-specific part └ department: "CS"	
Pointer to shared Person (vtable for virtual base)	← (internal implementation)

Midterm Exam

Date of the midterm exam



Please choose which date you prefer to have the midterm exam for programming course. [\[单选\]](#)

4.15

29票 47.5%

4.22

32票 52.5%

投票截至: 2027-04-08 12:00

吐个槽 | 举报投票

4.22 Afternoon

1-hour in class test

One piece of A4 cheat sheet